

Unified API Gateway & Key Management Platform

Technical Architecture & Implementation Specification

Document Version: 1.0

Date: July 4, 2026

Focus: High-Availability, LLM Streaming, and Agent-to-Agent Integration

1. Executive Summary

This document outlines the architecture for an API Key Management and Reverse Proxy platform offered as a resilient SaaS infrastructure. The system acts as a highly secure, low-latency intermediary layer between client applications and third-party upstream providers (e.g., OpenAI, Anthropic, Minima blockchain RPCs, and custom backend utilities).

Strategic Objective: To abstract credential management entirely from the client side, enforce strict Role-Based Access Control (RBAC), and capture granular usage analytics without exposing real provider credentials. This architecture is heavily optimized to support multi-agent orchestrators, high-volume data pipelines, and streaming LLM responses.

2. Core System Architecture

To handle high-throughput workloads securely, the system is decoupled into three distinct architectural components, ensuring strict isolation between the control plane and the data plane.

A. The Management Dashboard (Control Plane)

- **Role:** A frontend portal and administration backend.
- **Functionality:** Configure upstream providers, provision end-users, set granular budget alerts, and visualize time-series cost charts.

B. The Reverse Proxy (Data Plane)

- **Role:** A lightweight, stateless, ultra-fast routing engine built in a high-performance language like Go or Rust.
- **Functionality:** Intercepts incoming requests, validates custom proxy tokens, performs instantaneous authorization, and injects the actual API credentials before forwarding the payload. It natively handles Server-Sent Events (SSE) to ensure zero-latency degradation for AI inference.

C. Secure Vault & Analytics Engine

- **Security:** All upstream secret keys are encrypted using AES-256-GCM and stored in a secure enclave (e.g., HashiCorp Vault).
- **Telemetry:** Metrics are asynchronously published to a Kafka cluster and ingested by a time-series database (e.g., ClickHouse) to handle intensive event logging without blocking the proxy layer.

3. Database Schema Design

The relational core manages the state for tenants, users, and routing rules, while the time-series datastore handles high-velocity event logging.

Identity & Access Management (PostgreSQL)

- **Organizations**: `id`, `name`, `tier`, `created_at`
- **Users**: `id`, `org_id`, `email`, `role` (Admin, Developer, Agent)

Credential Routing Engine

- **Credentials**: `id`, `provider_name`, `encrypted_secret`, `monthly_budget_usd`, `status`
- **ProxyTokens**: `id`, `credential_id`, `public_token_hash`, `expires_at`
- **AccessRules**: `id`, `proxy_token_id`, `allowed_ips`, `allowed_models`, `rate_limit_rpm`

Audit & Analytics (ClickHouse / TimescaleDB)

- **UsageLogs**: `timestamp`, `proxy_token_id`, `status_code`, `path`, `prompt_tokens`, `completion_tokens`, `latency_ms`, `ip_address`

4. The Reverse Proxy Mechanism

The core value proposition relies on secure token masking. Client applications, whether they are mobile apps or autonomous agents in a registry, only interact with the proxy gateway.

Step-by-Step Request Lifecycle

1. **Client Request:** The application calls the proxy gateway, passing the provisioned proxy token.

```
POST https://gateway.yourplatform.com/v1/chat/completions
Authorization: Bearer xpxy_live_987654321
Content-Type: application/json

{
  "model": "gpt-4o",
  "messages": [{"role": "user", "content": "Initialize orchestrator."}]
}
```

1. **Authentication & Validation:** The proxy hashes `xpxy_live_987654321`, checks a Redis cache for validity, and verifies budget limits and rate constraints.
2. **Key Injection:** The proxy securely fetches the actual upstream API key, decrypts it in memory, and rewrites the authorization headers.
3. **Upstream Dispatch:** The request is routed to the target provider.

```
POST https://api.openai.com/v1/chat/completions
Authorization: Bearer sk-proj-RealSecretKeyHere
```

1. **Response Streaming:** The proxy streams chunks back to the client immediately. Upon completion, a background worker extracts the usage metrics and publishes them to the analytics queue.

5. Cost Optimization & Leak Detection

To ensure robust financial governance, the platform implements automated analysis of the telemetry stream.

- **Zombie Credential Detection:** A scheduled cron job cross-references active credentials against recent usage logs. Credentials with zero traffic over 30 days are flagged as optimization opportunities.
- **Geographic Anomaly Alerts:** The system maps IP addresses to locations. If a single proxy token is used in geographically impossible sequences (e.g., India and the US within 5 minutes), the token is automatically suspended.
- **Spike Thresholds:** By establishing a 7-day rolling baseline for each token, the system can detect when volume or cost metrics deviate by more than 3 standard deviations (3σ) within an hour, immediately alerting administrators to potential infinite loops or leaked tokens.